Tutorial for MOLE

based on commit 723d18b

Carlos Aznarán[†] Adelaida Otazu[‡]

March 9, 2025

[†]Universidad Nacional de Ingeniería
Av. Tupac Amaru, s/n, Lima, Perú
caznaranl@uni.pe
0000-0001-8314-2271
[‡]Universidad Nacional del Altiplano Puno
Avenida El Sol, Puno, Perú
aotazu@unap.edu.pe
0000-0003-4793-0400

Abstract

This document is written for newcomers in MOLE stands Mimetic Operators Library Enhanced, but with a solid foundation in numerical analysis for PDEs. The algorithms are continuously grows in size and quality and the examples are diverse implemented independently in GNU Octave/ MATLAB and C++ through Armadillo sparse linear algebra library. For now, these are exclusively the two language flavours. A clear comprenhension will gave you the capabilities to the translation into other high-performance scientific languages such as Julia, Python, Fortran, C and Rust.

Contents

I. The Foundations of the Mimetic Finite Difference Operators 1D **5**

1. Introduction and Objectives **7**

2. Mimetic Operadors List **8**
2.1. Divergence 8
2.2. Gradient 8
2.3. Interpolation from center to nodes 8
2.4. Interpolation from nodes to center 8
2.5. Laplacian 8

3. Numerical Methods for ODEs **9**
3.1. Backward Euler Method 9
3.2. Explicit Runge-Kutta 2 11
3.3. Explicit Runge-Kutta 4 12
3.4. Verlett 12

4. Numerical Solutions of Boundary Value Problems **14**

5. Numerical Methods for PDEs **15**
5.1. von Neumann Stability Criterion 15
5.2. Stability and Convergence Analysis 15
5.3. Method of characteristics 15
5.4. Method of separation of variables 17
5.5. Getting started with MOLE 17
5.5.1. Compiling and running the first code 17
5.6. Test 1 17
5.7. Test 2 17
5.8. Test 3 17
5.9. Test 4 17
5.10. Test 5 18

II. Numerical Exercises **19**

6. Foo **20**
6.1. Meeting MOLE 20
6.1.1. Create the staggered grid 20

6.2. Transport 1D 20
6.3. Poisson 1D 20
6.4. The 1D Diffusion Equation 25
6.5. Convection-diffusion 30

7. MOLE Tests **32**

A. Installing MOLE software on GNU/Linux **33**
A.1. Arch Linux 33
A.2. Ubuntu Linux 33

B. Documentation **35**
B.1. MOLE documentation 35
B.2. Programming language documentation 35
B.3. Linear Algebra software documentation 35
B.4. Extra documentation 36

NACIONAL DEL
UNIVERSIDAD
ALTIPLANO
PUNO

Part I.

The Foundations of the Mimetic Finite Difference Operators 1D

MOLE is an open-source library that implements high-order mimetic operators [7].

1. Introduction and Objectives

$$Gf_d = \vec{0}.$$

$$D\vec{v}_d = 0.$$

Let $U = [a, b]$.

$$\int_U \vec{v} \cdot \nabla f \,\mathrm{d}U + \int_U f \nabla \cdot \vec{v} \,\mathrm{d}U = \int_{\partial U} f \vec{v} \cdot \vec{n} \,\mathrm{d}S.$$

$$L^{(k)} = D^{(k)} G^{(k)}.$$

$$h\langle GF, V\rangle_P + h\langle DV, F\rangle_Q = F_{N+1}V_N - F_0 V_0 = V^T BF.$$

$$D^{(2)} = \frac{1}{h}\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1c} \\ \vdots & \ddots & \vdots \\ a_{r1} & \cdots & a_{rc} \end{bmatrix}, B = \begin{bmatrix} b_{11} & \cdots & b_{1q} \\ \vdots & \ddots & \vdots \\ b_{p1} & \cdots & a_{pq} \end{bmatrix} A \otimes B := \begin{bmatrix} a_{11}B & \cdots & a_{1c}B \\ \vdots & \ddots & \vdots \\ a_{r1}B & \cdots & a_{rc}B \end{bmatrix}.$$

$$D_{xy}^{(k)} = \begin{bmatrix} \widehat{I}_n \otimes D_x^{(k)} & D_y^{(k)} \otimes \widehat{I}_m \end{bmatrix}.$$

$$D_{xyz}^{(k)} = \begin{bmatrix} \widehat{I}_o \otimes \widehat{I}_n \otimes D_x^{(k)} & \widehat{I}_o \otimes D_y^{(k)} \otimes \widehat{I}_m & D_z^{(k)} \otimes \widehat{I}_n \otimes \widehat{I}_m \end{bmatrix}.$$

$$G_{xy}^{(k)} = \begin{bmatrix} \widehat{I}_n^T \otimes G_x^{(k)} \\ G_y^{(k)} \otimes \widehat{I}_m^T \end{bmatrix}.$$

$$G_{xyz}^{(k)} = \begin{bmatrix} \widehat{I}_o^T \otimes \widehat{I}_n^T \otimes G_x^{(k)} \\ \widehat{I}_o^T \otimes G_y^{(k)} \otimes \widehat{I}_m^T \\ G_z^{(k)} \otimes \widehat{I}_n^T \otimes \widehat{I}_m^T \end{bmatrix}.$$

2. Mimetic Operadors List

2.1. Divergence

2.2. Gradient

2.3. Interpolation from center to nodes

2.4. Interpolation from nodes to center

2.5. Laplacian

3. Numerical Methods for ODEs

Only the methods used in the tutorials: Verlett, RK4, Explicit Euler

Consider the Initial Value Problem

$$\begin{cases} \frac{\mathrm{d}y}{\mathrm{d}t} = f(t, y), & t \in [0, T]. \\ y(0) = y_0. \end{cases}$$

3.1. Backward Euler Method

$$f(t_{n+1}, y_{n+1}) = \frac{\mathrm{d}y}{\mathrm{d}t} \approx \frac{y_{n+1} - y_n}{\Delta t} \implies y_{n+1} = y_n + f(t_{n+1}, y_{n+1})\,\Delta t.$$

The local truncation error is $\mathcal{O}\left(\Delta t^2\right)$. The Butcher table is $\begin{array}{c|c} 1 & 1 \\ \hline & 1 \end{array}$.

$$\begin{cases} \frac{\mathrm{d}y}{\mathrm{d}t} = y \sin t^2, & t \in [0, 5]. \\ y(0) = 2. \end{cases}$$

$$S(x) := \int_0^x \sin\left(t^2\right)\,\mathrm{d}t = \sum_{n=0}^{\infty} \frac{(-1)^n x^{4n+3}}{(2n+1)!\,(4n+3)}.$$

Integramos y obtenemos la solución general.

$$\iint \frac{\mathrm{d}^2 u}{\mathrm{d}x^2}\,\mathrm{d}x\,\mathrm{d}x = \iint e^x\,\mathrm{d}x\,\mathrm{d}x.$$

$$\int \frac{\mathrm{d}u}{\mathrm{d}x}\,\mathrm{d}x = \int \left(e^x + C_1\right)\,\mathrm{d}x.$$

$$u(x) = e^x + C_1 x + C_2.$$

Ahora, apliquemos las condiciones de frontera Robin.

$$\begin{cases} 0 = u(0) - \left(\frac{\mathrm{d}u}{\mathrm{d}x}\right)_{x=0} = e^0 + C_1(0) + C_2 - \left(e^0 + C_1\right) = C_2 - C_1. \\ 2e = u(1) + \left(\frac{\mathrm{d}u}{\mathrm{d}x}\right)_{x=1} = e^1 + C_1(1) + C_2 + \left(e^1 + C_1\right) = 2e + 2C_1 + C_2. \end{cases} \tag{3.1}$$

El sistema (3.1) tiene como solución $C_1 = C_2 = 0$. $\therefore$ la solución de (6.1) es $u(x) = e^x$.

- `https://docs.octave.org/v9.4.0/Ranges.html`
- `https://docs.octave.org/v9.4.0/Solvers.html#XREFfzero`

```
#!/usr/bin/env -S octave -qf
% Solves ODE using backward Euler method

h = .05; % Step-size
t = 0:h:5; % Calculates up to y(5)
y = zeros(size(t));
y(1) = 2; % Initial condition
f = @(t, y) sin(t)^2 * y; % f(t, y)

for i = 1:length(t) - 1; % Stages
    old_y = y(i);
    y(i + 1) = fzero(@(y) y - h * f(t(i + 1), y) - old_y, old_y); % Backward Euler
end
```

Program 1: Program `backward_euler.m`

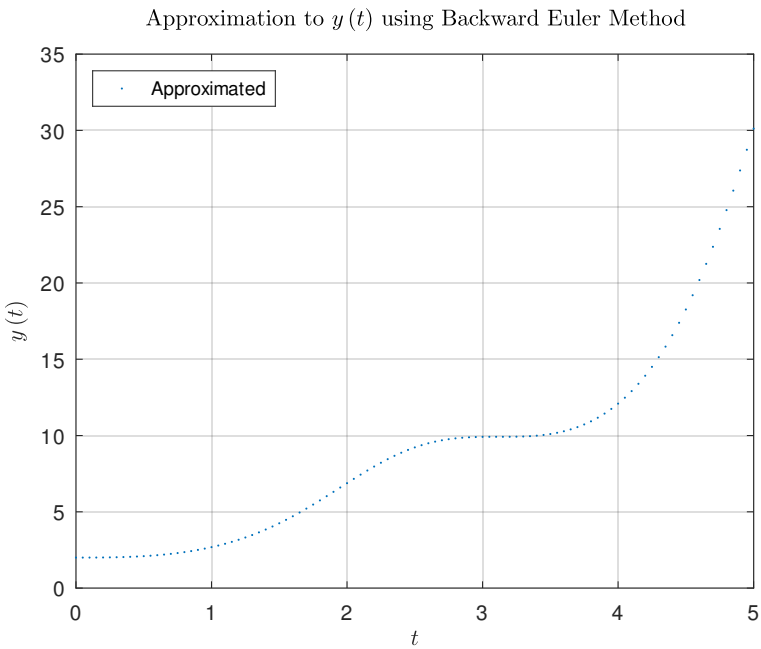

Figure 3.1.: Numerical solution by the Backward Euler Method.

- https://docs.octave.org/v9.4.0/Object-Sizes.html#XREFsize
- https://docs.octave.org/v9.4.0/Object-Sizes.html#XREFlength
- https://docs.octave.org/v9.4.0/Trigonometry.html#XREFsin
- https://docs.octave.org/v9.4.0/Special-Utility-Matrices.html#XREFzeros

3.2. Explicit Runge-Kutta 2

$$\widetilde{y}_{n+1} = y_n + f\left(t_n, y_n\right)\Delta t.$$
$$y_{n+1} = y_n + \frac{\Delta t}{2}\left(f\left(t_n, y_n\right) + f\left(\widetilde{y}_{n+1}, t_{n+1}\right)\right).$$

The local truncation error is $\mathcal{O}\left(\Delta t^2\right)$. The Butcher table is $\begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$.

```
#!/usr/bin/env -S octave -qf
% Solves ODE using explicit RK2 method

h = .05; % Step-size
t = 0:h:5; % Calculates up to y(5)
y = zeros(size(t));
y(1) = 2; % Initial condition
f = @(t, y) sin(t)^2 * y; % f(t, y)

for i = 1:length(t) - 1; % Stages
    k1 = f(t(i), y(i));
    k2 = f(t(i) + h / 2, y(i) + h / 2 * k1);
    y(i + 1) = y(i) + h * k2; % y(i + 1)
end
```

Program 2: Program `RK2.m`

```
//  RK2.cpp

// Description:
// This program solves a first-order ordinary differential equation (ODE) of the
// form:
//     dy/dt = sin^2(t) * y
// using the second-order Runge-Kutta (RK2) method. The solution is computed
// over the time interval [0, 5] with an initial condition y(0) = 2.0.

#include <armadillo>
#include <cmath>   // For sin()
#include <fstream> // For file output
#include <iomanip> // For setprecision
#include <iostream>
```

Program 3: Program `RK2.m`

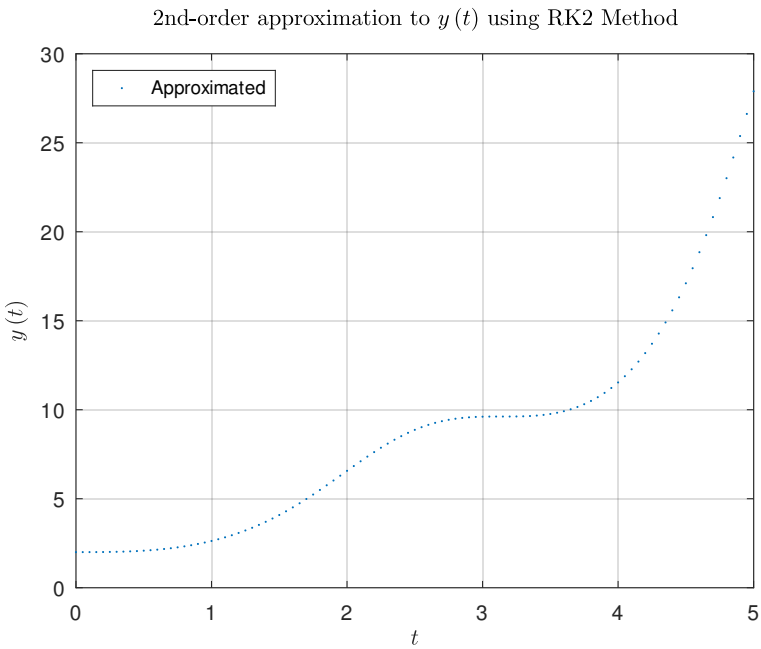

Figure 3.2.: Numerical solution by the Runge-Kutta 2 Method.

3.3. Explicit Runge-Kutta 4

$$
\begin{aligned}
k_1 &= f\left(t_n, y_n\right). \\
k_2 &= f\left(t_n + \frac{\Delta t}{2}, y_n + \Delta t \frac{k_1}{2}\right). \\
k_3 &= f\left(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta k_2}{2}\right). \\
k_4 &= f\left(t_n + \Delta t, y_n + \Delta t k_3\right). \\
y_{n+1} &= y_n + \frac{1}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right).
\end{aligned}
$$

The local truncation error is $\mathcal{O}\left(\Delta t^4\right)$. The Butcher table is $\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{4}{6} & \frac{4}{6} & \frac{1}{6} \end{array}$.

3.4. Verlett

```
#!/usr/bin/env -S octave -qf
% Solves ODE using explicit RK4 method

h = .05; % Step-size
t = 0:h:5; % Calculates up to y(5)
y = zeros(size(t));
y(1) = 2; % Initial condition
f = @(t, y) sin(t)^2 * y; % f(t, y)

for i = 1:length(t) - 1% Stages
    k1 = f(t(i), y(i));
    k2 = f(t(i) + h / 2, y(i) + h / 2 * k1);
    k3 = f(t(i) + h / 2, y(i) + h / 2 * k2);
    k4 = f(t(i) + h, y(i) + h * k3);
    y(i + 1) = y(i) + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4); % y(i + 1)
end
```

Program 4: Program `RK4.m`

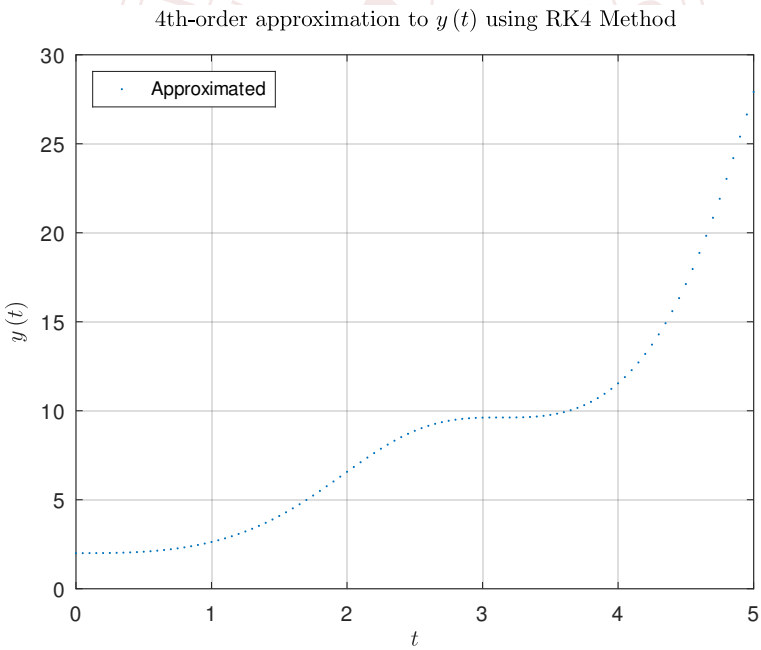

Figure 3.3.: Numerical solution by the Runge-Kutta 4 Method.

4. Numerical Solutions of Boundary Value Problems

5. Numerical Methods for PDEs

5.1. von Neumann Stability Criterion

5.2. Stability and Convergence Analysis

5.3. Method of characteristics

[5, 2]

Let's consider the problem of

$$\begin{cases} \partial_t u + c\partial_x u = 0, & x \in (0,1)\,,\ t > 0. \\ u(0,t) = u(1,t)\,, & t > 0. \\ u(x,0) = g(x)\,, & x \in [0,1]\,. \end{cases}$$

Consider the problem for the explicit form of linear first-oder PDEs in two independent variables

$$\begin{cases} a(x,y)\,\partial_x u + b(x,y)\,\partial_y u = c_1(x,y)\,u + c_2(x,y)\,, \\ u(x,y) \text{ given for } (x,y) \in \Gamma. \end{cases}$$

to be solved in some domain $\Omega \subset \mathbb{R}^2$ with data given on some curve $\Gamma \subset \overline{\Omega}$.

Often the $\Gamma \subset \partial\Omega \subset \mathbb{R}^2$ it will just be one of the coordinate axes.

We find the characteristics, i.e., the curves which follow these directions, by solving

$$\frac{\mathrm{d}x}{\mathrm{d}s} = a(x(s), y(s))\,, \qquad \frac{\mathrm{d}y}{\mathrm{d}s} = b(x(s), y(s))\,.$$

Now suppose u is a solution to the PDE. Let $z(s)$ denote the values of the solution u along a characteristic; i.e.,

$$z(s) \coloneqq u(x(s), y(s))\,.$$

Then by the chain rule, we have

$$\begin{aligned} \frac{\mathrm{d}z}{\mathrm{d}s} &= \partial_x u(x(s), y(s))\,\frac{\mathrm{d}x}{\mathrm{d}s}(x(s), y(s)) + \partial_y u(x(s), y(s))\,\frac{\mathrm{d}y}{\mathrm{d}s}(x(s), y(s))\,. \\ \frac{\mathrm{d}z}{\mathrm{d}s} &= \partial_x u(x(s), y(s))\,a(x(s), y(s)) + \partial_y u(x(s), y(s))\,b(x(s), y(s))\,. \\ \frac{\mathrm{d}z}{\mathrm{d}s} &= c_1(x(s), y(s))\,z(s) + c_2(x(s), y(s))\,. \end{aligned}$$

Definición 1: Characteristics equations

There are three *dependent variables* x, y and z and one *independent* variable s.

$$\begin{cases} \frac{\mathrm{d}x}{\mathrm{d}s}(s) &= a(x(s), y(s)). \\ \frac{\mathrm{d}y}{\mathrm{d}s}(s) &= b(x(s), y(s)). \\ \frac{\mathrm{d}z}{\mathrm{d}s}(s) &= c_1(x(s), y(s))\, z(s) + c_2(x(s), y(s)). \end{cases}$$

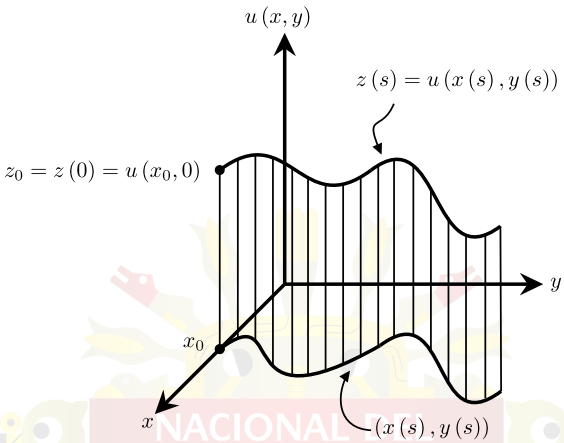

Figure 5.1.: The solution u is described by the surface defined by $z = u(x, y)$. From any point x_0 on the x-axis, there is a curve $(x(s), y(s))$ in the xy-plane, upon which wa can calculate the solution $z = u(x(s), y(s))$. Knowing only the structure of the PDE, x_0 and z_0 we can solve ODEs to find the part of the solution surface which lies above the curve.

$$a(x,y)\frac{\partial u}{\partial x} + b(x,y)\frac{\partial u}{\partial y} = c_1(x,y)\, u + c_2(x,y), u \text{ given for } (x,y) \in \Gamma$$

where we have a linear PDE in the independent variables x and y with a given functions a, b, c_1 and c_2 of (x, y). $\Gamma \subset \partial\Omega$. We find the characteristics, i.e., the curves which follow these directions, by solving

$$\begin{aligned} \frac{\mathrm{d}x}{\mathrm{d}s} &= a(x(s), y(s)). \\ \frac{\mathrm{d}y}{\mathrm{d}s} &= b(x(s), y(s)). \\ z(s) &= u(x(s), y(s)). \end{aligned}$$

$$\frac{\mathrm{d}z}{\mathrm{d}s} = c_1(x(s), y(s))\, z(s) + c_2(x(s), y(s)).$$

5.4. Method of separation of variables

5.5. Getting started with MOLE

5.5.1. Compiling and running the first code

5.6. Test 1

Program 5: Program `test1.cpp`

Program 6: Program `test1.m`

5.7. Test 2

Program 7: Program `test2.cpp`

Program 8: Program `test2.m`

5.8. Test 3

Program 9: Program `test3.cpp`

Program 10: Program `test3.m`

5.9. Test 4

Program 11: Program `test4.cpp`

Program 12: Program `test4.m`

5.10. Test 5

Program 13: Program `test5.cpp`

Program 14: Program `test5.m`

Part II.

Numerical Exercises

6. Foo

6.1. Meeting MOLE

The official website is `https://csrc-sdsu.github.io/mole`. After skimming the description and reading the papers you will find out that this method never uses a ghost points.

6.1.1. Create the staggered grid

6.2. Transport 1D

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} = 0.$$

```
{

  // Create a sparse matrix of size (m+1) x (m+1)
  arma::sp_mat S(m + 1, m + 1);
  if (type == "backward") {
    // Backward difference
    S.diag(-1) = -arma::ones<vec>(m);   // Sub-diagonal
    S.diag(0) = arma::ones<vec>(m + 1); // Main diagonal
    S(0, m - 1) = -1;                   // Wrap-around for periodic boundary
    S /= dx;
  }
  else if (type == "forward") {
    // Forward difference
    S.diag(0) = -arma::ones<vec>(m + 1); // Main diagonal
    S.diag(1) = arma::ones<vec>(m);      // Super-diagonal
    S(m, 1) = 1;                         // Wrap-around for periodic boundary
    S /= dx;
  }
  else { // "centered"
    // Centered difference
    S.diag(-1) = -arma::ones<vec>(m); // Sub-diagonal
    S.diag(1) = arma::ones<vec>(m);   // Super-diagonal
    S(0, m - 1) = -1;                 // Wrap-around for periodic boundary
    S(m, 1) = 1;                      // Wrap-around for periodic boundary
    S /= (2 * dx);
  }

```

Program 15: Programa `hyperbolic1Dupwind.cpp`

6.3. Poisson 1D

```
1  #!/usr/bin/env -S octave -qf
2  % Solves the 1D Poisson Equation with Robin Boundary Conditions and a
3  % non-constant forcing right hand side using the Mimetic Method with
4  % MOLE in Octave / MATLAB.
5  %
6  %     Δu = f
7  %
8  % u : Vertical Displacement of a membrane
9  % f : Forcing right hand side
10 % Δ : Laplace Operator
11
12 addpath('/usr/share/mole/matlab/')
13
14 west = 0; % Domain's limits
15 east = 1;
16
17 k = 6; % Operator's order of accuracy
18 m = 2 * k + 1; % Minimum number of cells to attain the desired accuracy
19 dx = (east - west) / m; % Step length
20
21 L = lap(k, m, dx); % 1D Mimetic laplacian operator
22 L_before_name = sprintf("L_before.h5");
23 save("-hdf5", L_before_name, "L")
24 figure('visible', 'off');
25 spy(L);
26 saveas(gcf, "elliptic1Dsparsebefore.pdf", 'pdfcrop')
27
28 % Impose Robin BC on laplacian operator
29 a = 1;
30 b = 1;
31 L = L + robinBC(k, m, dx, a, b);
32 L_after_name = sprintf("L_after.h5");
33 save("-hdf5", L_after_name, "L")
34 spy(L);
35 saveas(gcf, "elliptic1Dsparseafter.pdf", 'pdfcrop')
36
37 % 1D Staggered grid
38 grid = [west west + dx / 2:dx:east - dx / 2 east];
39 save("-hdf5", "grid")
40
41 % RHS
42 U = exp(grid)';
43 U(1) = 0; % West BC
44 U(end) = 2 * exp(1); % East BC
45 tic
46 U = L \ U; % Solve a linear system of equations
47 toc
48 save("-hdf5", "U")
49 % Plot result
50 plot(grid, U, 'o')
51 hold on
52 plot(grid, exp(grid))
53 legend('Approximated', 'Analytical', 'Location', 'NorthWest')
```

Program 16: Programa `elliptic1D.m`

$$
\begin{cases}
\dfrac{\mathrm{d}^2 u}{\mathrm{d}x^2} = e^x, \text{ para } x \in [0,1]\,. \\
0 = u(0) - \left(\dfrac{\mathrm{d}u}{\mathrm{d}x}\right)_{x=0}. \\
2e = u(1) + \left(\dfrac{\mathrm{d}u}{\mathrm{d}x}\right)_{x=1}.
\end{cases}
\tag{6.1}
$$

- En la línea 1 encontramos el *shebang*[1], esto permite ejecutar un script de Octave `./elliptic1D.m` con la opción de modo de procesamiento por lotes (batch), para

[1] `https://en.wikipedia.org/wiki/Shebang_(Unix)`

esto se necesita tener permisos de ejecución (por ejemplo, `chmod +x elliptic1D.m`).

- En las líneas 2 al 10 tenemos un comentario sobre el programa de modo que ayude al codificador a obtener un contexto del problema a resolver.

- En la línea 12, la función `addpath` agrega el directorio `"/usr/share/mole/matlab/"` a la ruta de búsqueda de la función. Allí se encuentran el conjunto de scripts Octave / MATLAB de la biblioteca MOLE. Vea el Programa **??**.

- En las líneas 14 y 15, se inicializan los identicadores `west` (oeste, izquierda), `east` (este, derecha) con los valores de 0 y 1, respectivamente, estos representan los valores de frontera del dominio espacial en (6.1).

- En la línea 21, llamamos a la función `lap`, este genera un operador Laplaciano discreto extendido que requiere como argumentos obligatorios el orden de precisión `k`, el número de celdas `m` y el tamaño de paso `dx`.

$$L = L^{(k)} = D^{(k)}G^{(k)} = DG. \qquad \left(\triangle = \nabla \cdot \nabla\right),$$

  donde D y G son los operadores miméticos de divergencia y gradiente, respectivamente. Dado que $D \in \mathbb{R}^{(m+2)\times(m+1)}$ y $G \in \mathbb{R}^{(m+1)\times(m+2)}$, entonces $L \in \mathbb{R}^{(m+2)\times(m+2)}$.

- En la línea 22, con la función `figure` desactivamos que se muestre la figura en la pantalla, preferimos solamente guardar la gráfica.

- En la línea 23, con la función `spy` graficamos (no se mostrará) el patrón de dispersidad de L.

- En la línea 24, con la función `saveas` guardamos esta gráfica en formato PDF y recortado.

- En la línea 29, llamamos a la función `robinBC`, este requiere como argumentos obligatorios el orden de precisión `k`, el número de celdas `m`, el tamaño de paso `dx`, el coeficiente Dirichlet `a` y el coeficiente Neumann `b`. Esta función devuelve una matriz en $\mathbb{R}^{(m+2)\times(m+2)}$. Actualizamos la matriz `L` según el Algoritmo 1.

Algorithm 1: Actualizaciones del operador Laplaciano discreto extendido.

1 $A \leftarrow L$;
2 $F \leftarrow f$;
3 $A \leftarrow A + R_G$;
4 $U \leftarrow \text{solve}\,(A, F)$;

- En la línea 34, creamos la malla escalonada unidimensional, note que los puntos internos son los centros de las celdas equiespaciados por `dx`. La distancia entre el extremo izquierdo y el posterior punto malla, así como del extremo derecho y el anterior punto malla es `dx/2`.

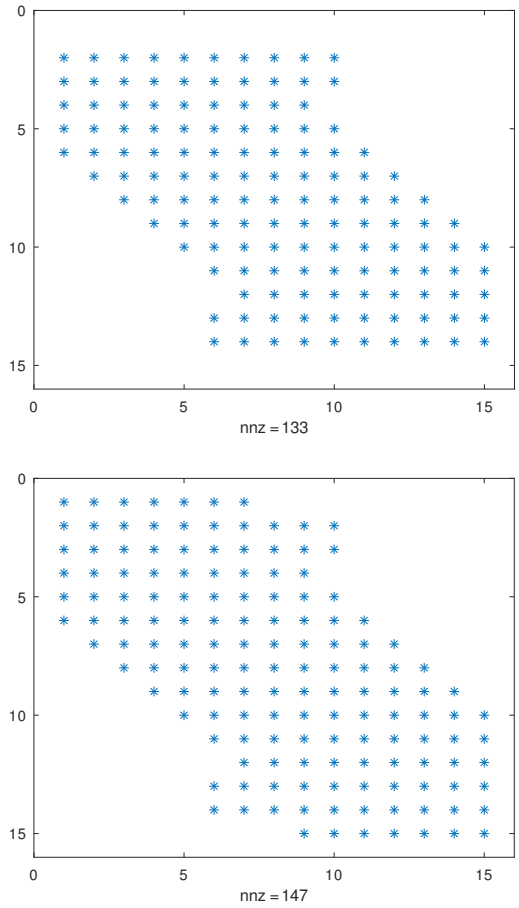

Figure 6.1.: Izquierda: Representación dispersa de L hasta la línea 21. La primera y la última fila son vectores de ceros. Derecha: Representación dispersa de L hasta la línea 29. La matriz $L \in \mathbb{R}^{15\times 15}$.

- En las líneas 35 y 43, guardamos `save` la malla computacional y la solución en el formato HDF5 para posterior post procesamiento.
- En la líneas 39 y 40, aplicamos las condiciones de frontera Robin, empleamos la función `exp`. El signo menos que antecede al coeficiente Neumann `b` se debe a que en el borde izquierdo de la malla el vector normal hacia afuera apunta hacia la izquierda, mientras que en el borde derecho el vector normal hacia afuera apunta hacia la derecha.
- En la línea 42, resolvemos el sistema de ecuaciones lineales disperso con la función `mldivide`.

Figure 6.2.: Diagrama de flujo del solucionador `mldivide` para matrices disperas que emplea MATLAB. Recuperado de `https://www.mathworks.com/help/matlab/ref/double.mldivide.html`.

Resultados del Programa 16

En primer lugar, mostramos la gráfica a escala 1:1 de la solución exacta y de la solución mimética obtenida en el Programa 16.

En segundo lugar, mostramos una gráfica del error en cada punto de la malla computacional dada por

$$\text{Error de } u \text{ en } x_{j-\frac{1}{2}} = \left| u\left(x_{j-\frac{1}{2}}\right) - u_{j-\frac{1}{2}} \right|.$$

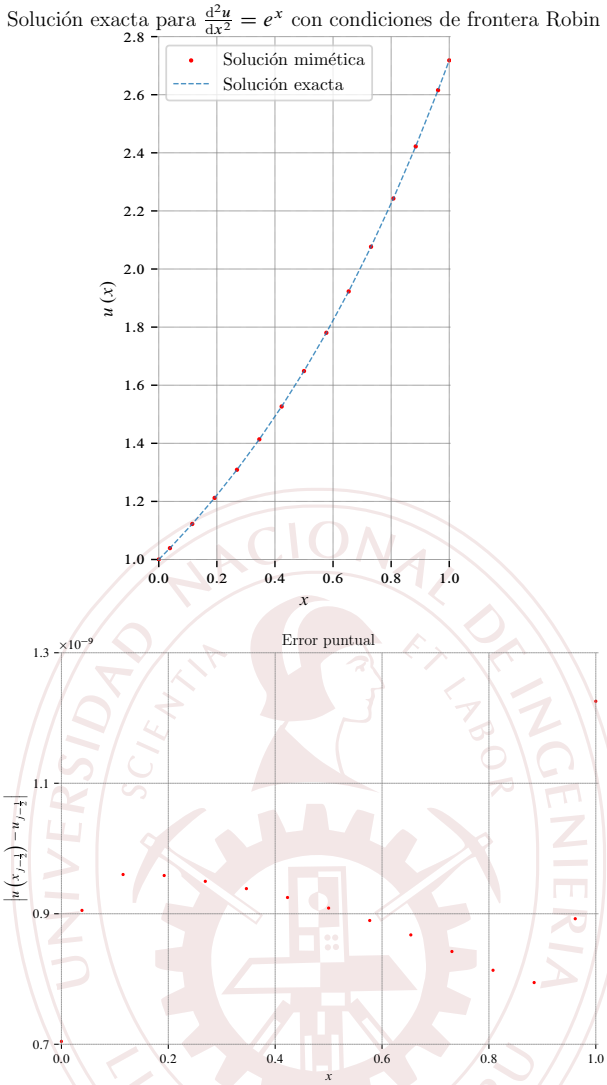

Figure 6.3.: Izquierda: Solución de (6.1) usando `k=6` y `m=2k+1=13`. Derecha: Error en la malla escalonada $\left\{0, \ldots, x_{j-\frac{1}{2}} \ldots, 1\right\}$

Por último, mostramos la tabla de los errores y el orden de convergencia numérico.

6.4. The 1D Diffusion Equation

Given the one-dimensional heat equation

Δx	Error ℓ_1	Orden
1.562×10^{-2}	4.410×10^{-2}	-
1.535×10^{-2}	4.464×10^{-2}	-0.678

Table 6.1.: Tabla de errores de aproximación de U en $x_{j-\frac{1}{2}}$ y el orden convergencia numérico obtenido.

$$\begin{cases} \frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}, & (x,t) \in (a,b) \times (0,\infty)\,. \\ u\,(x,0) = f(x), & x \in [a,b]\,. \\ u\,(a,t) = \alpha(t), & t \in (0,\infty)\,, \\ u\,(b,t) = \beta(t), & t \in (0,\infty)\,, \end{cases} \tag{6.2}$$

where κ is the thermal diffusivity.

The 1D heat equation code by mimetic methods

In the one-dimensional heat equation in the parabolic1D.m code, the PDE being solved is given by:

$$\begin{cases} \frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2}, & (x,t) \in (0,1) \times (0,1)\,. \\ u\,(x,0) = 0, & x \in [0,1]\,. \\ u\,(0,t) = 100., & t \in (0,1)\,, \\ u\,(1,t) = 100., & t \in (0,1)\,, \end{cases} \tag{6.3}$$

Line 9: In this part the code defines the value of the diffusion coefficient $\kappa = 1$.

Line 10: Define the left side $a = 0$ of the domain of the variable x.

Line 11: Define the right hand side $b = 1$ of the domain of the variable x.

Line 13: The Operator's order of accuracy $k = 2$.

Line 14: m is the number of cells, where m can take values $m \geq 2 * k + 1$; in this case $m = 2 * (2) + 1 = 5$.

Line 15: In this line the step size in x is defined, defined as $dx = (b - a)/m$, en this case, replacing it, we have $dx = \frac{1-0}{5} = \frac{1}{5}$

Line 17: End time $t = 1$.

Line 18: Von Neumann stability criterion for $k = 2$, we have $dt = \frac{(dx)^2}{3\kappa}$, in this exercise the step in time is given by: $dt = \frac{1}{75}$

```
#!/usr/bin/env -S octave -qf
% Solves the 1D Heat equation with Dirichlet boundary conditions

clc
close all

addpath('/usr/share/mole/matlab/')

alpha = 1; % Thermal diffusivity
west = 0; % Domain's limits
east = 1;

k = 2; % Operator's order of accuracy
m = 2*k+1; % Minimum number of cells to attain the desired accuracy
dx = (east-west)/m;

t = 1; % Simulation time
dt = dx^2/(3*alpha); % von Neumann stability criterion for explicit scheme, if k > 2 then /(4*alpha)

L = lap(k, m, dx); % 1D Mimetic laplacian operator

% IC
U = zeros(m+2, 1);
% BC
U(1) = 100;
U(end) = 100;

% 1D Staggered grid
grid = [west west+dx/2: dx :east-dx/2 east];

explicit = 1; % 0 = Implicit scheme

if explicit
    tic
    % Explicit
    L = alpha*dt*L + speye(size(L));

    % Time integration loop
    for i = 0 : t/dt+1
        plot(grid, U, 'o-')
        axis([0 1 0 105])
        str = sprintf('Explicit \t t = %.2f', i*dt);
        title(str)
        xlabel('x')
        ylabel('T')
        pause(0.01)
        U = L*U; % Apply the operator
    end
    toc
else
    tic
    % Implicit
    L = -alpha*dt*L + speye(size(L));
    dL=decomposition(L)
    %This line precomputes the LU decomposition of L and stores it as a
    %decomposition object. Because it's being stored as a decomposition
    %object. Matlab knows not to bother with LU factorizing L every time we
    %run \, which means that solving the system is sped up.
    %
    for i = 0 : t/dt+1
        plot(grid, U, 'o-')
        axis([0 1 0 105])
        str = sprintf('Implicit \t t = %.2f', i*dt);
        title(str)
        xlabel('x')
        ylabel('T')
        pause(0.01)
        U = dL\U;
    end
    toc
end
```

Program 17: Programa `parabolic1D.m`

Line 20: $L = lap(k, m, dx)$ 1D mimetic Laplacian operator is of order $m+2$ so $m+2$ for this exercise is of order 7 by 7, where $k = 2$, $m = 5$ and $\frac{1}{5}$, then $L = lap(2, 5, \frac{1}{5})$.

Line 23: The initial condition value $u(x, 0) = 0$ is a matrix of order $m + 2$ by 1, in this case 7 by 1.

Line 25: The boundary condition is on the left side u(a,t)= u(0,t)=100

Line 26: The boundary condition on the right side u(b,t)= u(1,t)=100

Line 29: The mesh used in the mimetic method grid $= [west\ west + dx/2 : dx : east - dx/2\ east]$, then grid $= [a\ a + \frac{dx}{2} : dx : b - \frac{dx}{2}\ b]$, in this exercise the mesh is given by:

$$\text{grid} = [0\ \tfrac{1}{10} : \tfrac{1}{5} : \tfrac{9}{10}\ 1] = \{0, \tfrac{1}{10}, \tfrac{3}{10}, \tfrac{5}{10}, \tfrac{7}{10}, \tfrac{9}{10}, 1\}$$

Line 31: If explicit=1 then the PDE will be solved in time by the explicit method, and if explicit=0 then the PDE will be solved by the implicit method.

Line 33 to 50: In this section we have the explicita solution of the PDE and the graph.

Line 36: The value of L is obtained by:

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2} \tag{6.4}$$

Discretizing $\frac{\partial u}{\partial t}$ by forward finite differences, and $\frac{\partial^2 u}{\partial x^2}$ by applying mimetic methods

$$\frac{u_i^{n+1} - u_i^n}{dt} = \kappa L u_i^n \tag{6.5}$$

then clearing u_i^{n+1} we obtain:

$$u_i^{n+1} = u_i^n + \kappa dt L u_i^n \tag{6.6}$$

Factoring u_i^n we have:

$$u_i^{n+1} = (I + \kappa dt L) u_i^n \tag{6.7}$$

where I is the identity matrix of general order $m + 2$ by $m + 2$, for this exercise of order 7 by 7, being

$$L = (\kappa dt L + I)$$

being in the code: $L = alpha * dt * L + speye(size(L))$; finally to obtain the solution

$$u_i^{n+1} = L * u_i^n \tag{6.8}$$

in the code on line 47 we have $U = L * U$.

Line 39: The iteration starts for $t = 0$ until $t = t/dt + 1$, in this exercise $t/dt + 1 = 75 + 1 = 76$.

Line 40 and 60: The graph of the mesh on the x axis by grid and the solution of the PDE U.

Line 41 and 62: In this line the graph is on the x axis from 0 to 1 and on the y axis from 0 to 105.

Line 42 and 63: In this line prints the different times $(i * dt)$for explicit method, with two decimal $(.2f)$.

Line 43 and 64: Use the title command to display the title designated on line 42.

Line 44 and 65: Label the x-axis in this exercise as x.
Line 45 and 66: Label the y-axis in this exercise as T.

Line 46 and 67: Shows the graph for the different times with a pause of 0.01.

Line 50 to 71: In this section we have the implicita solution of the PDE and the graph.

Line 53: The value of L is obtained by:

$$\frac{\partial u}{\partial t} = \kappa \frac{\partial^2 u}{\partial x^2} \tag{6.9}$$

Discretizing $\frac{\partial u}{\partial t}$ by forward finite differences, and $\frac{\partial^2 u}{\partial x^2}$ by applying mimetic methods

$$\frac{u_i^{n+1} - u_i^n}{dt} = \kappa L u_i^{n+1} \tag{6.10}$$

then clearing u_i^{n+1} we obtain:

$$u_i^{n+1}(I - \kappa dt L) = u_i^n \tag{6.11}$$

where I is the identity matrix of general order $m + 2$ by $m + 2$, for this exercise of order 7 by 7, being

$$L = (-\kappa dt L + I)$$

being in the code: $L = -alpha * dt * L + speye(size(L))$.

Line 54: In this line $dL = \text{decomputation}(L)$ the matrix L is decomposed into lower triangular matrix L and upper triangular matrix U using LU decomposition method.

Line 68: In this line solve the $u^{n+1} = (dL)^{-1} * u^n$, thus finding the solution $U = dL \backslash U$.

6.5. Convection-diffusion

Given the three-dimensional convection diffusion equation

$$\begin{cases} \frac{\partial u}{\partial t} + \nabla(vu) = \nabla \cdot (D\nabla u), & (x, y, z, t) \in (a, b) \times (c, d) \times (e, f) \times (0, \infty)\,. \\ u\,(x, 0) = f(x), & x \in [a, b]\,. \\ u\,(a, t) = \alpha(t), & t \in (0, \infty)\,, \\ u\,(b, t) = \beta(t), & t \in (0, \infty)\,, \end{cases} \tag{6.12}$$

NACIONAL DEL
UNIVERSIDAD
ALTIPLANO
PUNO

```
#!/usr/bin/env -S octave -qf
% convection_diffusion.m | Solves convection-diffusion equation using MOLE

clc
close all
format short

addpath('/usr/share/mole/matlab/')

% Mimetic operator's parameters
k = 2;
m = 101;
n = 51;
o = 101;

% Domain's dimensions
a = 0;
b = 101;
c = 0;
d = 51;
e = 0;
f = 101;

% Spatial step sizes
dx = (b - a) / m;
dy = (d - c) / n;
dz = (f - e) / o;

% Mimetic operators
D = div3D(k, m, dx, n, dy, o, dz);
G = grad3D(k, m, dx, n, dy, o, dz);
I = interpol3D(m, n, o, 1, 1, 1);

% Pore velocity vector
V = zeros(size(G, 1), 1);
% Density vector
C = zeros(m + 2, n + 2, o + 2);

% Impose initial conditions ----------------------------------------
bottom = 10; % Well
top = 15; % Well
seal = 40; % Shale

% Velocity field ---------------------------------------------------
y = ones(m, n + 1, o);
y(:, seal, :) = 0;
y(:, seal + 5, :) = 0;
y = y(:);
V(((m + 1) * n * o + 1):((m + 1) * n * o + numel(y))) = y; % Shale

% Density ----------------------------------------------------------
C(ceil((m + 2) / 2), bottom:top, ceil((o + 2) / 2)) = 1; % ceil((o+2)/2)
C = C(:);
idx = find(C);

% Diffusivity and porosity -----------------------------------------
diff = 1; %[m^2/s] CO2 molecular diffusivity at: 45°C, S = 35, and P = 75atm
porosity = 1; % Sandstone -> 30%
diff = diff * porosity;
K = diff * ones(size(G, 1), 1);
kk = diff * ones(m, n + 1, o);
kk(:, seal, :) = diff / 10;
kk(:, seal + 5, :) = diff / 40;
kk = kk(:);
K(((m + 1) * n * o + 1):((m + 1) * n * o + numel(kk))) = kk; % Shale
% ------------------------------------------------------------------

% dt based on von Neumann criterion
dt1 = dx^2 / (3 * diff) / 3;
% dt based on CFL condition
dt2 = (dx / max(V)) / 3;
% Select minimum dt
dt = min(dt1, dt2);

iters = 120; % 90 = 30s  if dt = 0.3333 because CFL

% Premultiplication of Laplacian operator
L = dt * D * spdiags(K, 0, numel(K), numel(K)) * G;
L = L + speye(size(L));
```

Program 18: Programa `convection_diffusion.m`

7. MOLE Tests

UNIVERSIDAD NACIONAL DEL ALTIPLANO
PUNO

A. Installing MOLE software on GNU/Linux

To work through this tutorial requires to have a working installation of MOLE. It relies on `armadillo` [12], a C++ library that provides data structures for sparse matrices. We explain the step-by-step process for Arch Linux and Ubuntu Linux, as both systems have been sucessfully tested by us.

A.1. Arch Linux

This distribution is supported by a proactive group of developers, package maintainers and support staff that try to provides the latest stable software releases. The steps are outlined in the Program 19.

```bash
#!/bin/bash

# [1 / 3] Update the system and install developer tools
sudo pacman --needed --noconfirm -Syu base-devel git
# [1.5 / 3] Optionally install Intel MKL as a replacement for LAPACK linear algebra library
sudo pacman -S intel-oneapi-mkl # or depending on your needs: intel-oneapi-basekit, intel-oneapi-hpckit, lapack, blas-openblas

# [2 / 3] Install armadillo from https://aur.archlinux.org/armadillo.git
git clone https://aur.archlinux.org/armadillo.git
pushd armadillo
makepkg -s --noconfirm
popd

# [3 / 3] Install MOLE C++/Octave from https://aur.archlinux.org/libmole.git
git clone https://aur.archlinux.org/libmole.git
pushd libmole
makepkg -s --noconfirm
popd
```

Program 19: Steps for a system-wide installation both C++ and Octave MOLE library vía `installerarchlinux.sh`.

Even if you are using Windows, the Docker Desktop WSL 2 backend is ideal for using MOLE via Program 20 or installing Arch Linux on WSL 2 and following the Program 19.

A.2. Ubuntu Linux

This Debian-derived distribution is managed by Canonical Ltd. Each 2 years they launch a Long Term Support(LTS) release. The steps are outlined in the Program 21.

```bash
#!/bin/bash

docker run -it --rm ghcr.io/carlosal1015/mole_examples/libmole-git # or libmole instead of libmole-git
docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
ghcr.io/carlosal1015/mole_examples/libmole-git         latest    a86fb0fef044   3 hours ago    2.16GB
ghcr.io/carlosal1015/mole_examples/libmole             latest    d20a1ec0091b   3 hours ago    2.14GB
```

Program 20: Pull container based on Arch Linux with set up MOLE library vía `docker.sh`.

```bash
#!/bin/bash

sudo apt-get update
sudo apt-get --no-install-recommends --yes install \
    build-essential cmake git octave \
    libarmadillo-dev libsuperlu-dev libeigen3-dev libgtest-dev
sudo apt-get install --reinstall ca-certificates

git clone --filter=blob:none --depth=1 https://github.com/csrc-sdsu/mole.git
sed -i '/^if(POLICY/,+51 s/^/#/' mole/CMakeLists.txt
sed -i '/^set(LINK_LIBS/,+3 s/^/#/' mole/CMakeLists.txt
sed -i '96i set(LINK_LIBS ${ARMADILLO_LIBRARIES} ${OpenBLAS_LIBRARIES} ${LAPACK_LIBRARY}
↪  ${SUPERLU_INSTALL_DIR}/lib/x86_64-linux-gnu/libsuperlu.so)' mole/CMakeLists.txt
sed -i '/^include(/,+9 s/^/#/' mole/tests/cpp/CMakeLists.txt

cmake \
    -S mole \
    -B build \
    -DBUILD_SHARED_LIBS=TRUE \
    -DCMAKE_BUILD_TYPE=None \
    -DCMAKE_CXX_STANDARD=14 \
    -DCMAKE_CXX_COMPILER=g++ \
    -DCMAKE_INSTALL_PREFIX=/usr \
    -Wno-dev
cmake --build build --target mole_C++
cmake --build build --target install
```

Program 21: Steps for a system-wide installation both C++ and Octave MOLE library vía `installerubuntu.sh`.

B. Documentation

We split the documentation in three categories.

B.1. MOLE documentation

General docs `https://carlosal1015.github.io/mole_examples/html`

C++ docs `https://carlosal1015.github.io/mole_examples/cpp/html`

Octave docs `https://carlosal1015.github.io/mole_examples/matlab`

B.2. Programming language documentation

- C++ docs

```
#include <iostream>
#include <cmath>
#include <vector>
```

- Octave docs

```
addpath("/usr/share/")
```

- MATLAB docs

B.3. Linear Algebra software documentation

Intel MKL docs `https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl-documentation.html`

Openblas docs `http://www.openmathlib.org/OpenBLAS/docs`

Netlib Lapack docs `https://www.netlib.org/lapack/explore-html`

SuperLU docs `https://portal.nersc.gov/project/sparse/superlu/superlu_code_html/index.html`

Eigen docs `https://eigen.tuxfamily.org/dox`

Armadillo docs `https://arma.sourceforge.net/docs.html`

B.4. Extra documentation

Gtest docs https://google.github.io/googletest

SciPy sparse docs https://docs.scipy.org/doc/scipy/reference/sparse.html

Matplotlib docs https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.plot.html

H5DF for Python docs https://docs.h5py.org/en/stable

References

[1] Daniele Andreucci et al. "Some Numerical Results on Chemotactic Phenomena in Stem Cell Therapy for Cardiac Regeneration". In: *Mathematics* 12.13 (2024). ISSN: 2227-7390. DOI: 10.3390/math12131937. URL: https://www.mdpi.com/2227-7390/12/13/1937.

[2] Daniel Arrigo. *An Introduction to Partial Differential Equations*. Cham: Springer International Publishing, 2023. ISBN: 978-3-031-22087-6.

[3] Jared Brzenski. "Building an ocean model using mimetic operators". Doctoral Dissertation. San Diego: San Diego State University, 2024. URL: https://digitalcollections.sdsu.edu/do/af38107a-44b2-444e-b504-16fa8ec4cf53.

[4] Jared Brzenski and Jose E. Castillo. "Solving Navier-Stokes with mimetic operators". In: *Computers & Fluids* 254 (2023), p. 105817. ISSN: 0045-7930. DOI: https://doi.org/10.1016/j.compfluid.2023.105817.

[5] Rustum Choksi. *Partial Differential Equations: A First Course*. American Mathematical Society, Apr. 2022. ISBN: 978-1-4704-6491-2.

[6] Johnny Corbino and Jose E. Castillo. "High-order mimetic finite-difference operators satisfying the extended Gauss divergence theorem". In: *Journal of Computational and Applied Mathematics* 364 (2020), p. 112326. ISSN: 0377-0427. DOI: 10.1016/j.cam.2019.06.042.

[7] Johnny Corbino, Miguel A. Dumett, and Jose E. Castillo. "MOLE: Mimetic Operators Library Enhanced". In: *Journal of Open Source Software* 9.99 (2024), p. 6288. DOI: 10.21105/joss.06288. URL: https://doi.org/10.21105/joss.06288.

[8] Yessica Judith Gonzales Aredo. "Convergencia del método mimético para la ecuación de difusión no estática". MA thesis. Trujillo: Universidad Nacional de Trujillo, 2023. URL: https://dspace.unitru.edu.pe/items/1742d2ae-7727-4c4b-9bf9-0cadbbb75d3c.

[9] Konstantin Lipnikov, Gianmarco Manzini, and Mikhail Shashkov. "Mimetic finite difference method". In: *Journal of Computational Physics* 257 (2014). Physics-compatible numerical methods, pp. 1163–1227. ISSN: 0021-9991. DOI: https://doi.org/10.1016/j.jcp.2013.07.031.

[10] Bertha K. Rodriguez-Chavez and Yessica E. Zarate-Pedrera. "Spectral Differentiation and Mimetic Methods for Solving the Scalar Burger's Equation". In: *Selecciones Matemáticas* 11.02 (Dec. 2024), pp. 259–270. DOI: 10.17268/sel.mat.2024.02.05. URL: https://revistas.unitru.edu.pe/index.php/SSMM/article/view/6157.

[11] Franco Rubio López and Mardo Gonzales Herrera. "Algorítmo para la Ecuación de Difusión en Estado Estacionario 2D usando el Método Mimético en Diferencias Finitas." In: *Selecciones Matemáticas* 1.01 (Apr. 2015). DOI: 10.17268/sel.mat.2014.01.04. URL: https://revistas.unitru.edu.pe/index.php/SSMM/article/view/826.

[12] Conrad Sanderson and Ryan Curtin. *Armadillo: An Efficient Framework for Numerical Linear Algebra*. 2025. DOI: 10.48550/arXiv.2502.03000. arXiv: 2502.03000 [cs.MS].

[13] G. Sosa Jones, J. Arteaga, and O. Jiménez. "A study of mimetic and finite difference methods for the static diffusion equation". In: *Computers & Mathematics with Applications* 76.3 (2018), pp. 633–648. ISSN: 0898-1221. DOI: 10.1016/j.camwa.2018.05.004.

[14] Angel Boada Velazco. "High order mimetic finite differences on non-trivial problems". Doctoral Dissertation. San Diego: San Diego State University, 2021. URL: https://digitalcollections.sdsu.edu/do/e67f17cb-b906-4847-a574-a8781e581024.